



**KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN MẠNG VÀ CÁC HỆ THỐNG THÔNG TIN**

CHƯƠNG 3

TIỀN TRÌNH



Nội dung chính - Chương 3

Mục lục

- 3.1. Tiến trình và trạng thái tiến trình
- 3.2. Thao tác trên tiến trình
- 3.3. Mô tả tiến trình
- 3.4. Luồng

Từ khóa

Process	Program
State	PCB
Context Switch	PID
fork()	exec()
Thread	Multithreading

Trọng tâm kiến thức



Hiểu mô hình tiến trình

Phân biệt chương trình và tiến trình; mô tả cấu trúc vùng nhớ và các trạng thái.

Theo dõi vòng đời

Giải thích tạo, thực thi, chờ và kết thúc tiến trình trong UNIX/Linux.

Phân tích luồng

Nhận diện tài nguyên dùng chung, dữ liệu riêng và các mô hình ánh xạ luồng.

Câu hỏi định hướng: Hệ điều hành lưu giữ và chuyển giao trạng thái thực thi như thế nào để nhiều công việc cùng tiến triển an toàn?



3.1. Tiến trình và trạng thái tiến trình

Tiến trình là một chương trình đang thực thi cùng trạng thái và tài nguyên liên quan



Chương trình là thụ động, tiến trình là chủ động

Chương trình

Tập mã lệnh nằm trên thiết bị lưu trữ; tự nó không thực hiện công việc và không có trạng thái CPU.

Tiến trình

Một lần thực thi cụ thể của chương trình, có bộ đếm lệnh, thanh ghi, vùng nhớ, tệp mở và tài nguyên được cấp.

Một chương trình trở thành tiến trình khi được nạp vào bộ nhớ và bắt đầu được CPU thực thi.



Một tiến trình có mã lệnh, dữ liệu, heap và stack



Text/code chứa mã lệnh; data chứa biến toàn cục; heap cấp phát động; stack giữ lời gọi hàm, biến cục bộ và địa chỉ trả về.



Một chương trình có thể tạo nhiều tiến trình độc lập

Cùng mã lệnh

Nhiều lần chạy có thể dùng chung tệp thực thi hoặc vùng mã chỉ đọc.

Trạng thái riêng

Mỗi tiến trình có PID, bộ đếm lệnh, thanh ghi, stack và dữ liệu đang thay đổi riêng.

Tài nguyên riêng

Không gian địa chỉ và bảng tệp mở được quản lý độc lập, trừ phần được chia sẻ có chủ ý.

Hai cửa sổ của cùng một ứng dụng vẫn có thể là hai tiến trình với hai chuỗi thực thi khác nhau.



Năm trạng thái cơ bản của tiến trình

Trạng thái

New — đang được tạo

Ready — sẵn sàng, chờ CPU

Running — đang thực thi

Waiting — chờ I/O hoặc sự kiện

Terminated — đã kết thúc

Chuyển trạng thái

admit

dispatch

interrupt

preemption

I/O wait

event wait

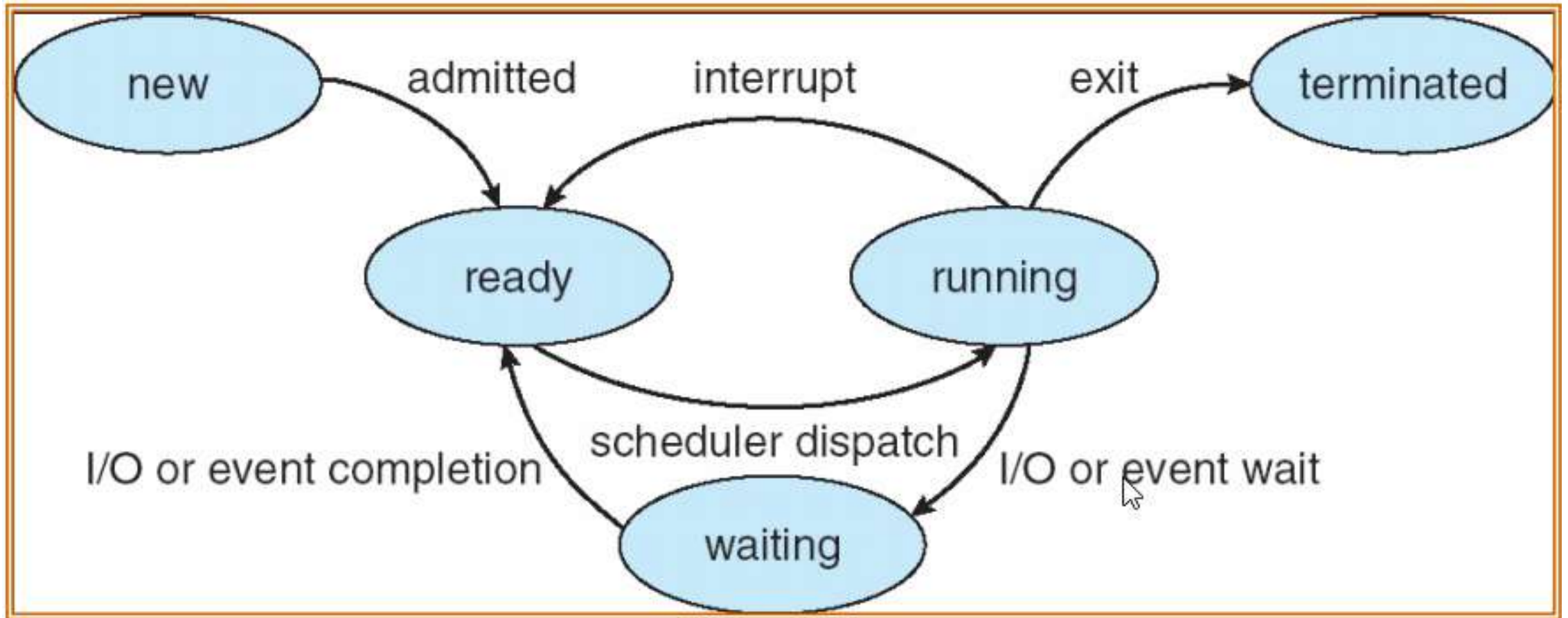
I/O done

event done

exit

reclaim

Trạng thái thay đổi theo hoạt động hiện tại





Mỗi chuyển trạng thái phản ánh một sự kiện hệ thống

Ready ↔ Running

Bộ lập lịch cấp CPU để chuyển Ready sang Running; ngắt hoặc tước quyền đưa tiến trình đang chạy trở lại Ready.

Running ↔ Waiting

Yêu cầu I/O hoặc chờ sự kiện làm tiến trình bị chặn; khi sự kiện hoàn tất, tiến trình quay lại Ready, không đi thẳng vào Running.

New được thừa nhận vào hệ thống; Running chuyển sang Terminated khi thực thi xong hoặc bị kết thúc.



Phân biệt Running, Ready và Waiting

Running

Đang sở hữu một lõi CPU và thực thi lệnh tại thời điểm xét.

Ready

Có đủ tài nguyên cần thiết, chỉ còn chờ được bộ lập lịch cấp CPU.

Waiting

Chưa thể chạy dù CPU rỗi vì còn chờ I/O, tín hiệu hoặc một sự kiện khác.

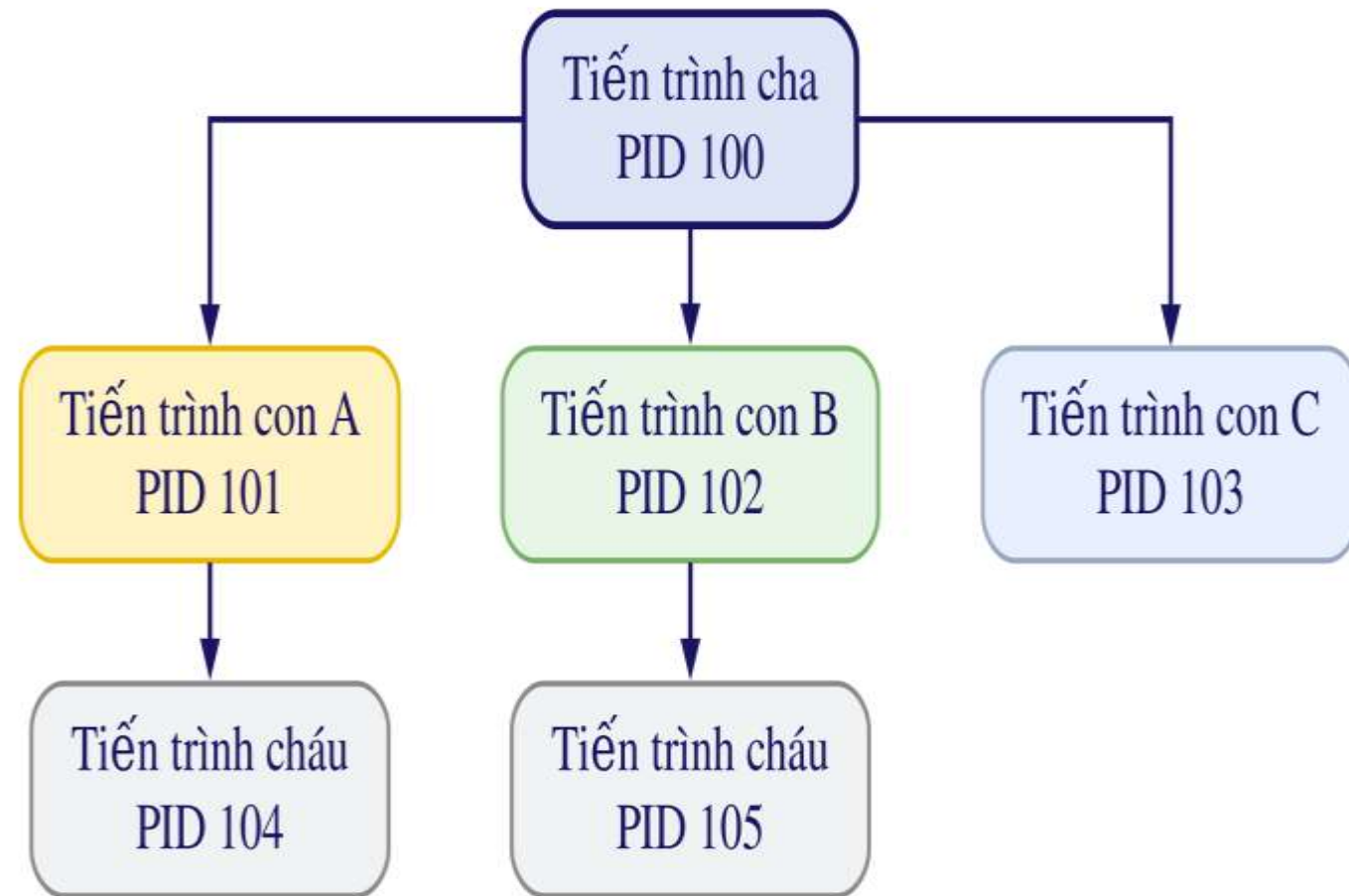
Tại một thời điểm, mỗi lõi chỉ chạy tối đa một luồng; nhiều tiến trình khác có thể ở Ready hoặc Waiting.



3.2. Thao tác trên tiến trình

Hệ điều hành cung cấp cơ chế tạo, nạp chương trình, chờ và kết thúc tiến trình

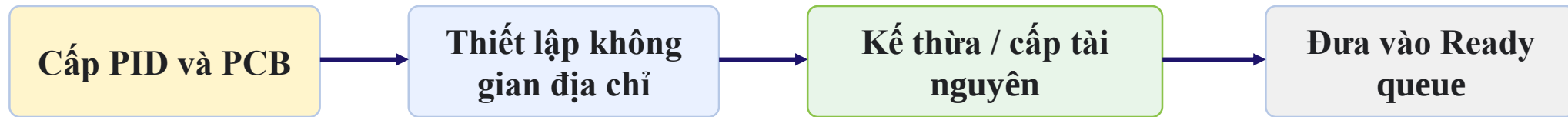
Tiến trình tạo ra quan hệ cha - con





Bốn bước tạo tiến trình

Hệ điều hành phải tạo thực thể quản lý trước khi tiến trình mới có thể tham gia hàng đợi sẵn sàng.



Tiến trình cha và con có thể chạy đồng thời; cha cũng có thể chờ con hoàn tất tùy logic chương trình.



Quan hệ cha - con gồm tài nguyên và thứ tự thực thi

Chia sẻ tài nguyên

Có thể chia sẻ toàn bộ, một phần hoặc không chia sẻ, tùy cơ chế và hệ điều hành.

Không gian địa chỉ

Con thường nhận ảnh bản đầu từ cha; cơ chế copy-on-write trì hoãn sao chép vật lý đến khi có ghi.

Quan hệ thực thi

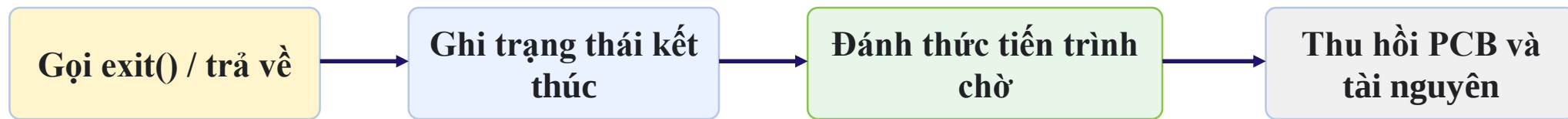
Cha và con chạy đồng thời hoặc cha chờ con; thứ tự không cố định nếu không có đồng bộ.

PID xác định tiến trình; PPID ghi nhận tiến trình cha và hỗ trợ biểu diễn cây tiến trình.



Kết thúc tiến trình phải đi kèm thu hồi tài nguyên

Tiến trình có thể tự kết thúc bình thường, gặp lỗi hoặc bị một tiến trình có quyền yêu cầu chấm dứt.



Trong UNIX/Linux, cha dùng wait() hoặc waitpid() để nhận trạng thái và hoàn tất việc thu hồi thông tin của tiến trình con.



Các tình huống kết thúc cần phân biệt

Kết thúc bình thường

Tiến trình hoàn thành, trả mã trạng thái và giải phóng tài nguyên.

Bị kết thúc

Cha hoặc hệ điều hành dừng tiến trình do lỗi, vượt giới hạn hoặc nhiệm vụ không còn cần.

Zombie

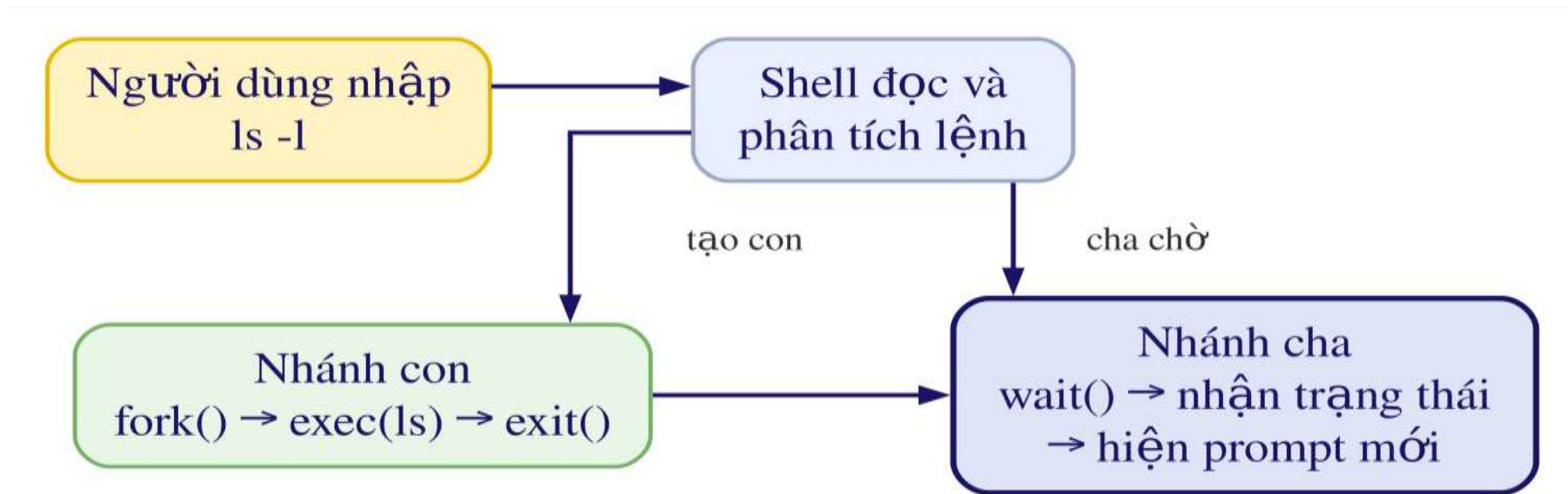
Con đã kết thúc nhưng cha chưa wait(); còn giữ bản ghi tối thiểu về PID và trạng thái.

Orphan / dây chuyền

Cha kết thúc trước con; hệ thống nhận nuôi hoặc áp dụng cơ chế kết thúc theo dây chuyền.



Ví dụ: shell thực thi một lệnh ngoài



Shell vẫn là tiến trình cha. Tiến trình con thay ảnh bằng chương trình cần chạy; khi con kết thúc, shell tiếp tục và hiển thị dấu nhắc mới.



3.3. Mô tả tiến trình

Hệ điều hành dùng PCB để lưu mọi thông tin cần thiết nhằm quản lý và tiếp tục thực thi tiến trình



Hệ điều hành cần một bản mô tả cho từng tiến trình

Nhận diện

Phân biệt tiến trình, người sở hữu và quan hệ cha - con bằng các mã định danh.

Tiếp tục thực thi

Khôi phục đúng vị trí lệnh, thanh ghi, stack và trạng thái sau khi tiến trình tạm dừng.

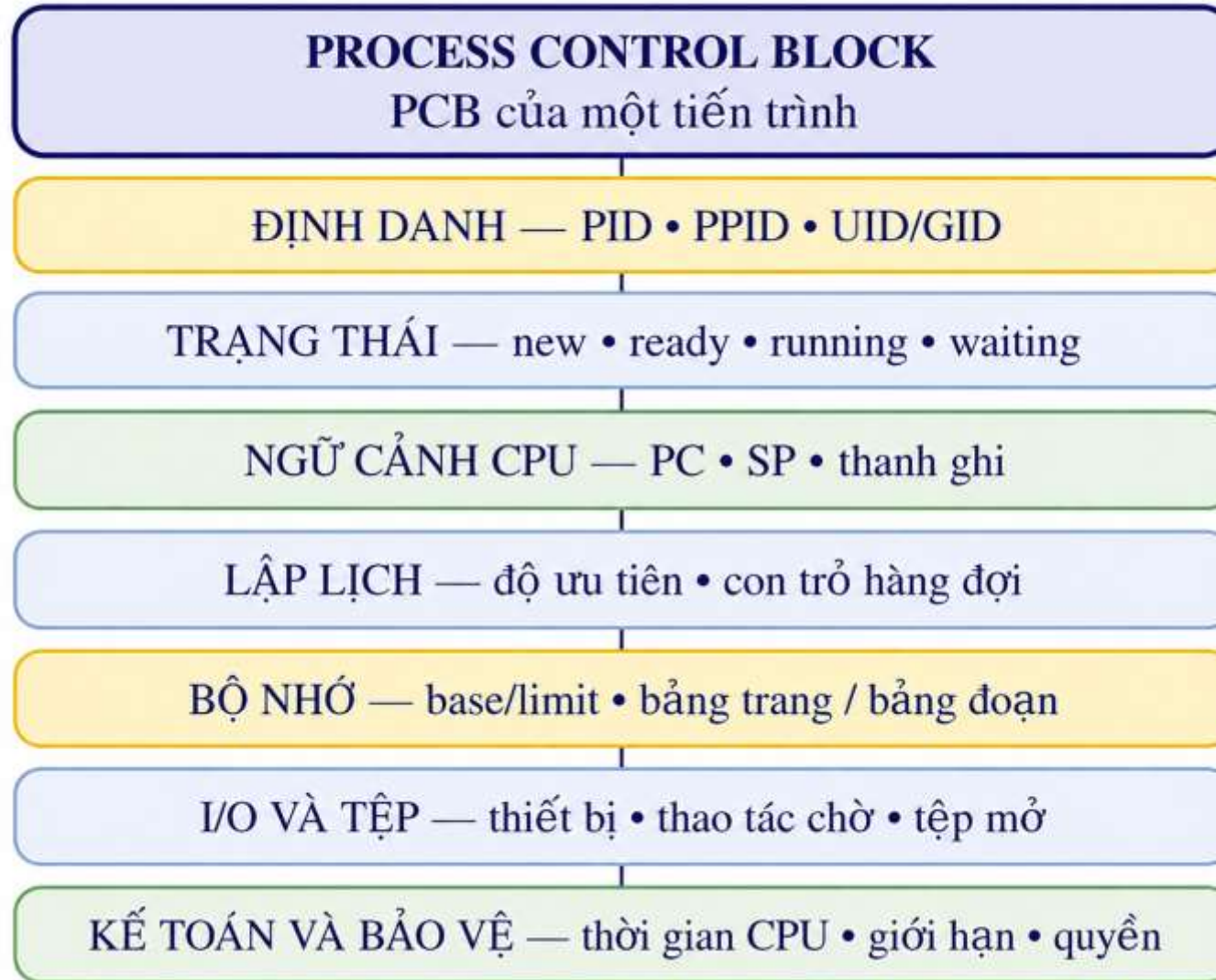
Quản lý tài nguyên

Theo dõi bộ nhớ, tệp, I/O, quyền, thông tin lập lịch và mức sử dụng tài nguyên.

PCB là bản ghi trong vùng nhớ của nhân; nó không phải là phần mã lệnh của chương trình người dùng.



PCB là kho thông tin thay đổi theo từng tiến trình





Định danh và trạng thái cho biết tiến trình là ai, đang ở đâu

Định danh

PID, PPID, UID/GID và các con trỏ quan hệ cho phép nhân tra cứu, kiểm tra quyền và tổ chức cây tiến trình.

Trạng thái

New, Ready, Running, Waiting hoặc Terminated; kèm nguyên nhân chờ và các cờ điều khiển cần thiết.

Mỗi thay đổi trạng thái phải được cập nhật nhất quán trong PCB và cấu trúc hàng đợi liên quan.



Ngữ cảnh CPU giúp tiến trình tiếp tục đúng chỗ

Bộ đếm chương trình và thanh ghi

Program Counter chỉ lệnh tiếp theo; các thanh ghi đa dụng, thanh ghi trạng thái và con trỏ stack giữ kết quả trung gian.

Thông tin bộ nhớ

Thanh ghi base/limit, bảng trang hoặc bảng đoạn mô tả không gian địa chỉ mà tiến trình được phép sử dụng.

Khi ngắt hoặc chuyển đổi ngữ cảnh, nhân phải lưu đủ trạng thái kiến trúc để kết quả thực thi không bị thay đổi.



PCB còn chứa thông tin lập lịch và tài nguyên

Lập lịch CPU

Độ ưu tiên, tham số lập lịch, thời gian CPU và con trỏ liên kết trong hàng đợi.

I/O và tệp

Thiết bị được cấp, thao tác đang chờ, bảng mô tả tệp và danh sách tệp đang mở.

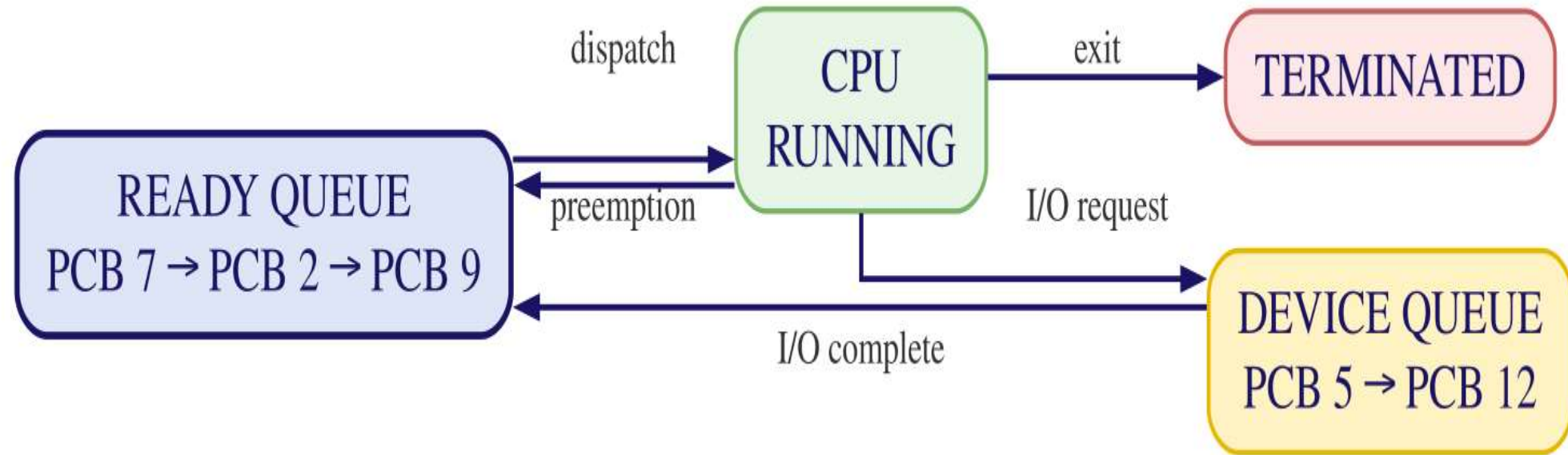
Kế toán và bảo vệ

Thời gian sử dụng, giới hạn tài nguyên, thông tin chủ sở hữu, quyền và tín hiệu.

Nội dung PCB phụ thuộc hệ điều hành và kiến trúc, nhưng mục tiêu chung là quản lý đầy đủ vòng đời tiến trình.



Các hàng đợi liên kết những PCB có cùng điều kiện chờ



Ready queue chứa các PCB chỉ chờ CPU. Device queue chứa các PCB chờ thiết bị hoặc I/O; khi sự kiện hoàn tất, PCB được chuyển về Ready queue.

Chuyển đổi ngữ cảnh đổi CPU từ tiến trình này sang tiến trình khác



Lưu ngữ cảnh cũ

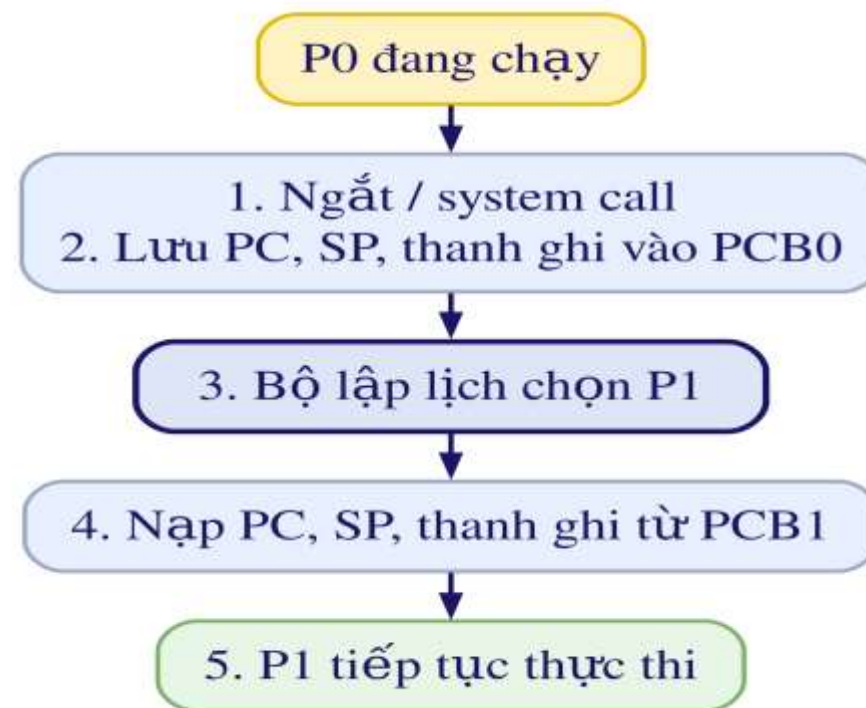
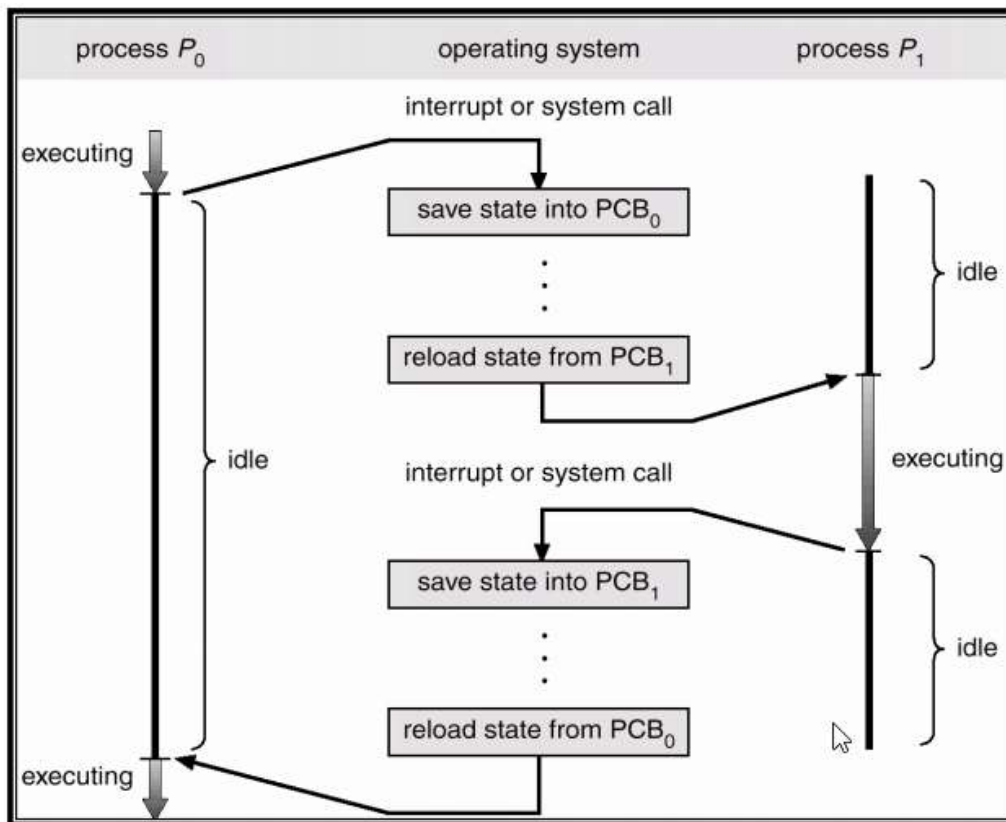
Nhân lưu Program Counter, Stack Pointer, thanh ghi và thông tin trạng thái của tiến trình đang chạy vào PCB tương ứng.

Khôi phục ngữ cảnh mới

Nhân nạp trạng thái từ PCB của tiến trình được chọn, cập nhật không gian địa chỉ rồi trả CPU về user mode.

Thời gian chuyển đổi ngữ cảnh là chi phí quản lý: Hệ thống đang chuyển trạng thái, chưa thực hiện công việc hữu ích của ứng dụng.

Chuyển đổi ngữ cảnh phải bảo toàn chuỗi thực thi



Sau khi được lập lịch lại, P_0 phải quan sát trạng thái như thể nó chỉ tạm dừng tại đúng điểm trước đó; mọi thanh ghi cần thiết phải được khôi phục chính xác.



Thời gian chuyển đổi phụ thuộc mức trạng thái cần thay

Lưu và nạp trạng thái

Số lượng thanh ghi, trạng thái SIMD/FPU và cơ chế phần cứng ảnh hưởng thời gian chuyển đổi.

Bộ nhớ và cache

Đổi không gian địa chỉ có thể làm giảm hiệu quả TLB và cache, tạo chi phí gián tiếp.

Tần suất chuyển đổi

Chuyển quá thường xuyên tăng tính đáp ứng nhưng làm tăng phần thời gian dành cho quản lý.

Chuyển giữa các luồng cùng tiến trình thường nhẹ hơn vì chúng chia sẻ không gian địa chỉ và nhiều tài nguyên.



Ảnh tiến trình và PCB bổ sung cho nhau

Ảnh tiến trình

Không gian người dùng gồm code, data, heap và stack; chứa chương trình và dữ liệu mà tiến trình trực tiếp thao tác.

PCB trong nhân

Siêu dữ liệu quản lý, ngữ cảnh CPU và liên kết hệ thống; chỉ nhân truy cập trực tiếp để lập lịch, bảo vệ và thu hồi.

Ảnh tiến trình trả lời “tiến trình đang chứa gì”; PCB trả lời “hệ điều hành quản lý và tiếp tục tiến trình như thế nào”.



Lập lịch tiến trình

Tận dụng tối đa CPU trong đa chương trình

Chu kỳ của các CPU-I/O burst – Việc thực thi tiến trình là một chu kì của các thực thi bởi CPU và chờ đợi vào ra

Phân phối CPU burst

Điều kiện lập lịch

- | | |
|---|----------------|
| - Tính tận dụng CPU: Thông thường từ 40-90% | ⇒ Cực đại hóa |
| - Thông lượng | ⇒ Cực đại hóa |
| - Thời gian quay vòng (turnaround time) | ⇒ Cực tiểu hóa |
| - Thời gian chờ | ⇒ Cực tiểu hóa |
| - Thời gian phản ứng | ⇒ Cực tiểu hóa |

Lập lịch tiến trình



Tiến trình	AT	BT
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Giải thích đề

AT (Arrival Time) cho biết tiến trình xuất hiện lúc nào.

BT (Burst Time) cho biết tiến trình cần bao nhiêu đơn vị CPU.

Khi lập biểu đồ Gantt, chỉ cần xác định thứ tự chạy và các mốc thời gian.

Tính thời gian chờ trung bình (Ttb) bằng công thức chung.

Mục tiêu duy nhất: tính thời gian chờ trung bình (Ttb)



Lập lịch tiến trình

FCFS: phục vụ theo thứ tự đến

First-Come, First-Served

Ý tưởng

Tiến trình nào đến trước thì chạy trước.

Không chiếm đoạt: đã chạy thì chạy đến hết CPU burst.

Cách làm đơn giản, nhưng tiến trình ngắn có thể phải chờ lâu.

Quy tắc tính

Sắp xếp theo AT tăng dần.

Ghi thời điểm bắt đầu và kết thúc từng tiến trình.

CT (Completion Time) là mốc kết thúc của tiến trình trên Gantt.

$WT = CT - AT - BT$. (WT: Wait Time)

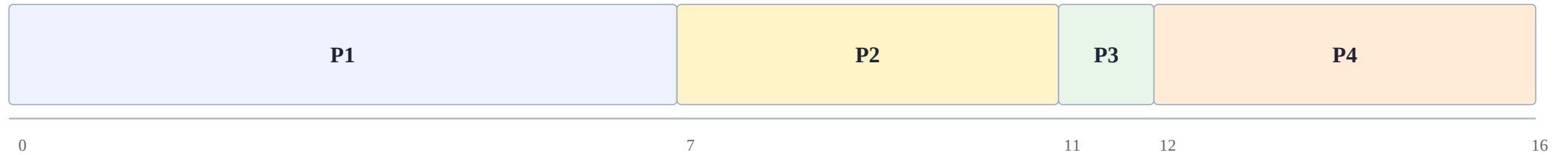
FCFS không cần so sánh BT khi chọn tiến trình kế tiếp



Lập lịch tiến trình

FCFS: hướng dẫn tính thời gian chờ

Ví dụ với P1, P2, P3, P4



P	AT	BT	CT	WT = CT-AT-BT
P1	0	7	7	0
P2	2	4	11	5
P3	4	1	12	7
P4	5	4	16	7

$$T_{tb} = (0 + 5 + 7 + 7) / 4 = 4,75$$



Lập lịch tiến trình

SJF không chiếm đoạt: chọn tiến trình ngắn nhất

Shortest Job First - Nonpreemptive

Khi CPU rảnh

Chọn tiến trình đã đến có BT nhỏ nhất.

Đang chạy

Không bị ngắt giữa chừng, dù có tiến trình ngắn hơn xuất hiện.

Kết quả thường gặp

Giảm thời gian chờ trung bình nếu biết trước BT.

Tại mỗi lần chọn: chỉ xét các tiến trình đã đến và chưa hoàn thành

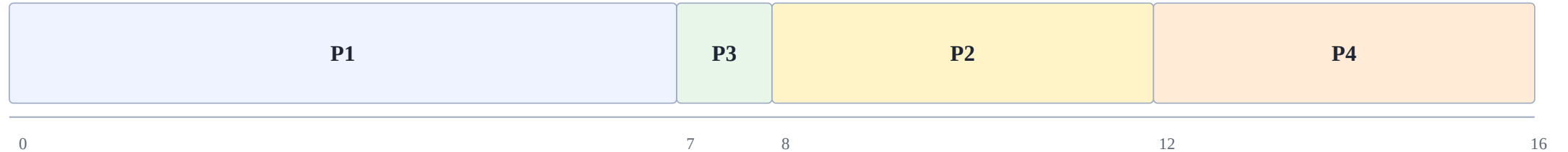
Nếu bằng BT, có thể ưu tiên tiến trình đến trước để bài tính nhất quán.



Lập lịch tiến trình

SJF không chiếm đoạt: hướng dẫn tính

Chọn tiến trình ngắn nhất mỗi khi CPU rảnh



P	AT	BT	CT	WT
P1	0	7	7	0
P2	2	4	12	6
P3	4	1	8	3
P4	5	4	16	7

Sau P1, các tiến trình P2, P3, P4 đều đã đến; chọn P3 vì BT = 1 nhỏ nhất.

$$T_{tb} = (0 + 6 + 3 + 7) / 4 = 4$$



Lập lịch tiến trình

SJF chiếm đoạt: luôn chọn thời gian còn lại ngắn nhất

Shortest Remaining Time First

Ý tưởng

Mỗi khi có tiến trình mới đến, so sánh thời gian còn lại.

Nếu tiến trình mới ngắn hơn, tiến trình đang chạy bị tạm dừng.

Còn gọi là SRTE.

Cách làm

Duyệt theo các mốc tiến trình đến và mốc hoàn thành.

Luôn chọn tiến trình có remaining time nhỏ nhất.

CT là thời điểm tiến trình chạy xong hoàn toàn.

WT vẫn tính bằng CT - AT - BT, không cộng từng đoạn chờ thủ công



Lập lịch tiến trình

SJF chiếm đoạt: hướng dẫn tính

Ví dụ có tạm dừng và chạy tiếp



P	AT	BT	CT	WT
P1	0	7	16	9
P2	2	4	7	1
P3	4	1	5	0
P4	5	4	11	2

P1 chạy trước, nhưng bị dừng khi P2 đến. P3 đến lúc 4 và có thời gian còn lại ngắn nhất.

$$T_{tb} = (9 + 1 + 0 + 2) / 4 = 3$$

Lập lịch tiến trình



Round Robin: chia CPU theo lát thời gian

Mỗi tiến trình nhận một quantum q

Hàng đợi sẵn sàng

Tiến trình đến sau được đưa vào cuối hàng đợi.

Quantum

Nếu chưa xong sau q đơn vị, tiến trình về cuối hàng đợi.

Công bằng

Phù hợp hệ thống tương tác, nhưng kết quả phụ thuộc q .

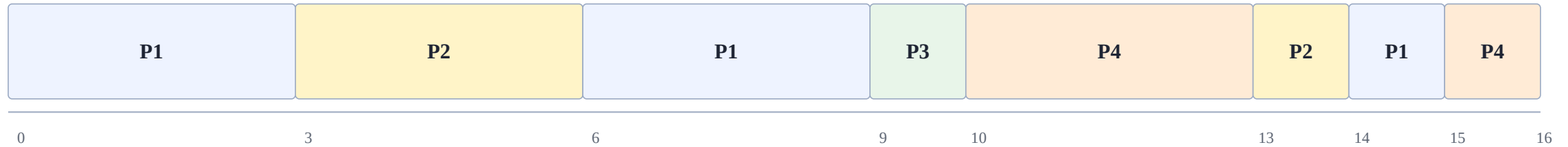
Trong ví dụ: chọn $q = 3$

Lập lịch tiến trình



Round Robin: hướng dẫn tính thời gian chờ

Quantum $q = 3$



P	AT	BT	CT	WT
P1	0	7	15	8
P2	2	4	14	8
P3	4	1	10	5
P4	5	4	16	7

Chỉ cần lấy CT ở lần cuối cùng mỗi tiến trình xuất hiện trên Gantt.

$$T_{tb} = (8 + 8 + 5 + 7) / 4 = 7$$



Lập lịch tiến trình

So sánh kết quả trên cùng dữ liệu

Chỉ xét thời gian chờ trung bình

Thuật toán	Thứ tự / đặc điểm	Ttb
FCFS	Theo thứ tự đến	4,75
SJF không chiếm đoạt	Chọn BT nhỏ nhất khi CPU rảnh	4
SJF chiếm đoạt	Chọn remaining time nhỏ nhất	3
Round Robin	Chia CPU theo quantum $q = 3$	7

Với bộ dữ liệu này, SJF chiếm đoạt cho WTtb nhỏ nhất

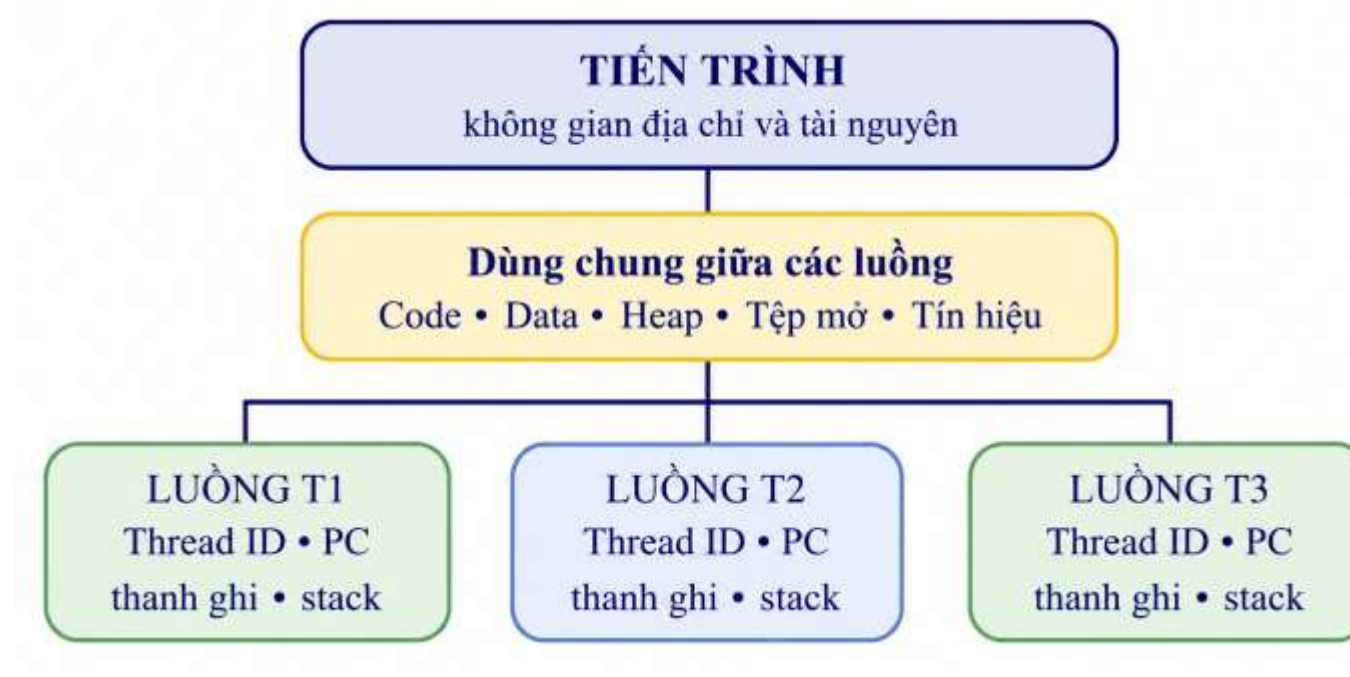


3.4. Luồng

Luồng là đơn vị sử dụng CPU bên trong tiến trình; các luồng chia sẻ tài nguyên nhưng có ngữ cảnh thực thi riêng



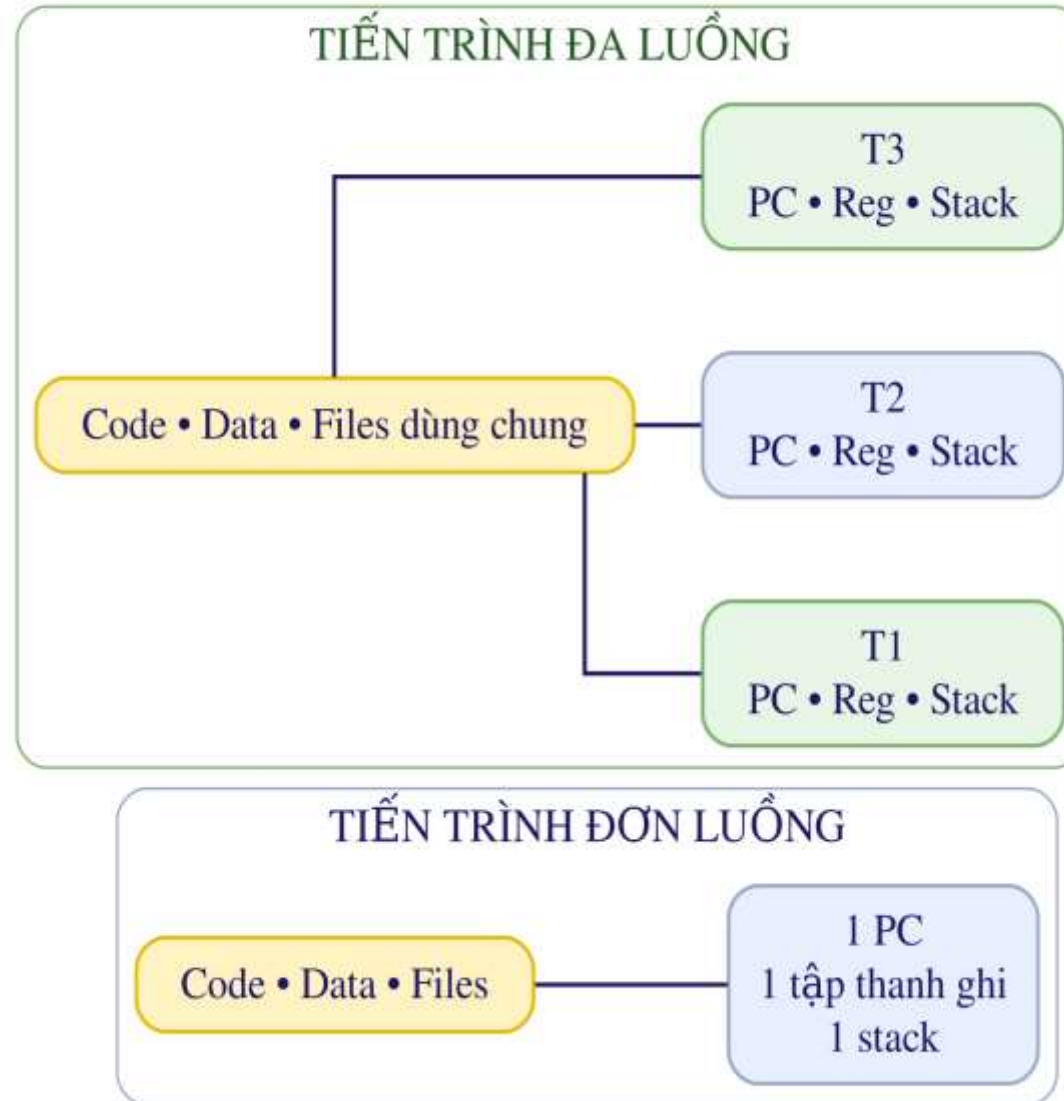
Mỗi luồng có ID, PC, thanh ghi và stack riêng



Các luồng trong cùng tiến trình dùng chung code, data, heap, tệp mở và phần lớn tài nguyên hệ điều hành; vì vậy trao đổi dữ liệu thuận tiện nhưng cần đồng bộ.



Tiến trình đa luồng chứa nhiều chuỗi thực thi





Đa luồng hỗ trợ nhiều hoạt động đồng thời

Trình duyệt

Một luồng hiển thị giao diện, luồng khác tải dữ liệu mạng hoặc xử lý nội dung.

Trình soạn thảo

Luồng nhận phím, luồng hiển thị, luồng lưu tự động và kiểm tra chính tả nền.

Máy chủ

Một luồng nhận kết nối; nhiều luồng phục vụ các yêu cầu độc lập của khách hàng.

Nhân hệ điều hành cũng thường dùng nhiều luồng cho I/O, quản lý bộ nhớ, xử lý ngắt và các dịch vụ nền.



Bốn lợi ích chính của đa luồng

Tính đáp ứng

Ứng dụng vẫn phản hồi khi một luồng bị chặn hoặc thực hiện tác vụ dài.

Chia sẻ tài nguyên

Các luồng mặc định chia sẻ code, dữ liệu và tệp của tiến trình.

Tính kinh tế

Tạo và chuyển luồng thường nhẹ hơn tạo và chuyển tiến trình.

Khả năng mở rộng

Nhiều luồng có thể chạy song song trên các lõi xử lý khác nhau.



Luồng mức người dùng và luồng mức nhân

User-level thread

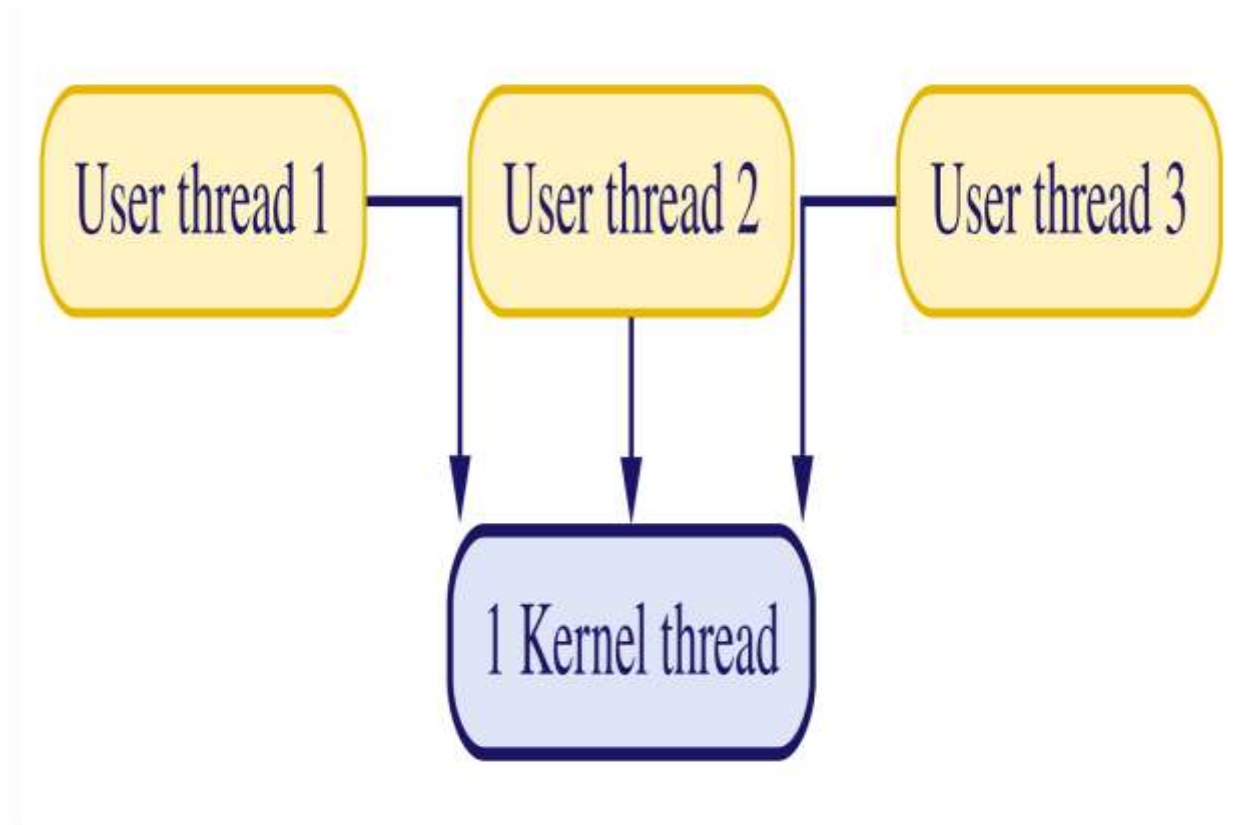
Thư viện ở không gian người dùng quản lý.
Tạo/chuyển nhanh và linh hoạt, nhưng nhân có thể không nhìn thấy từng luồng.

Kernel-level thread

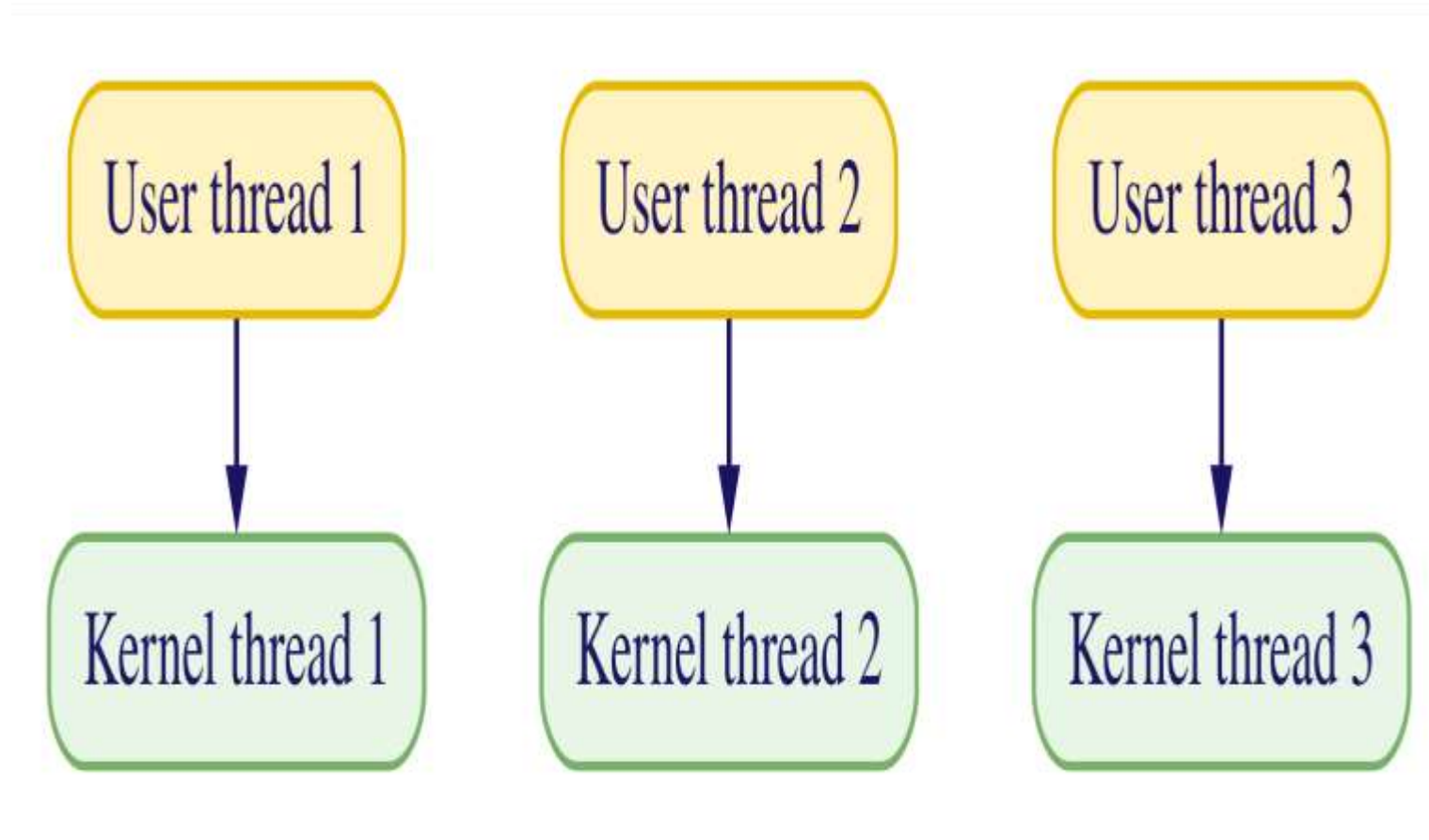
Hệ điều hành quản lý và lập lịch trực tiếp. Hỗ trợ chặn độc lập và song song đa lỗi, nhưng thao tác thường tốn chi phí hơn.

Mô hình đa luồng xác định cách các luồng người dùng được ánh xạ lên các luồng mà nhân có thể lập lịch.

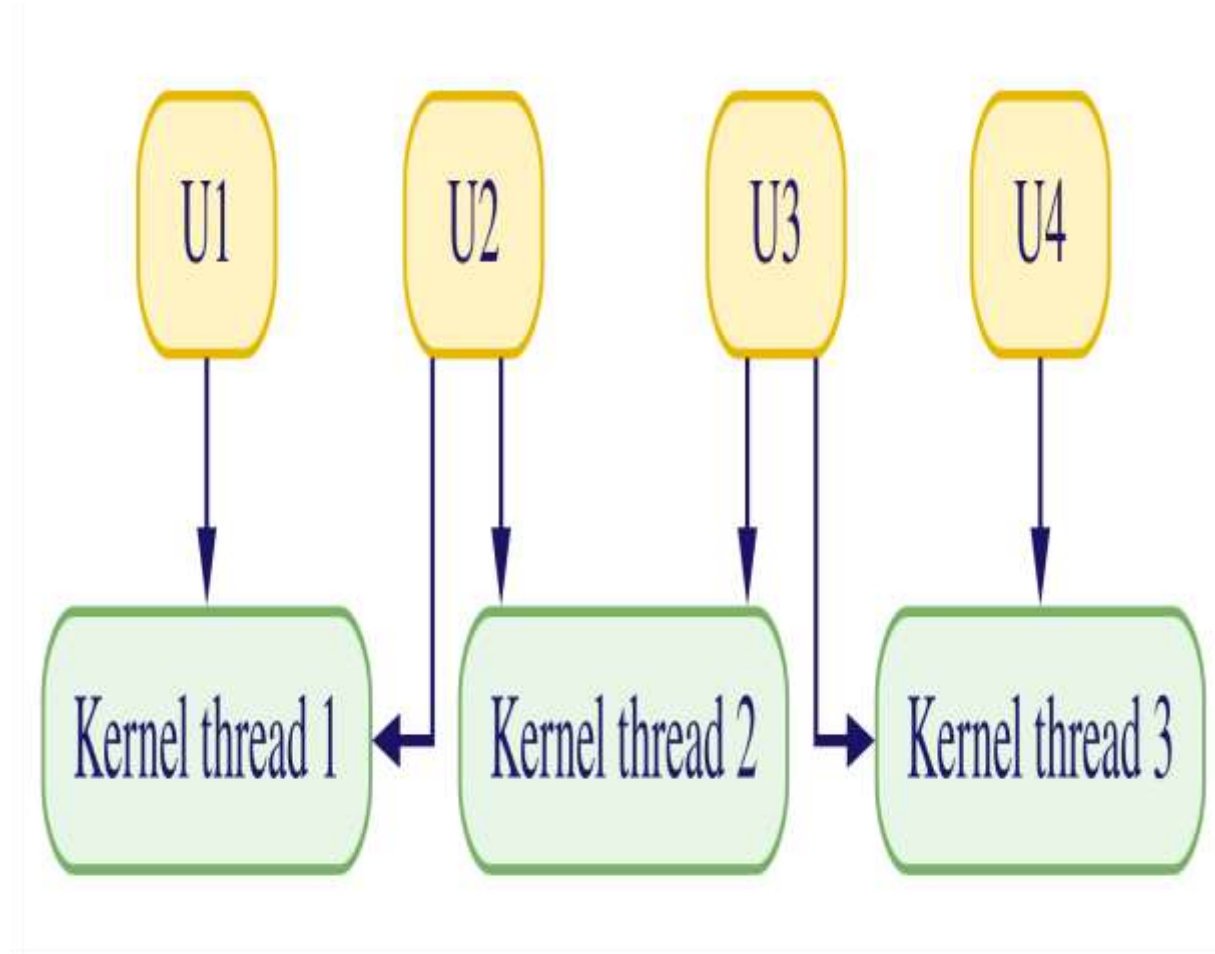
Mô hình Many-to-One



Mô hình One-to-One

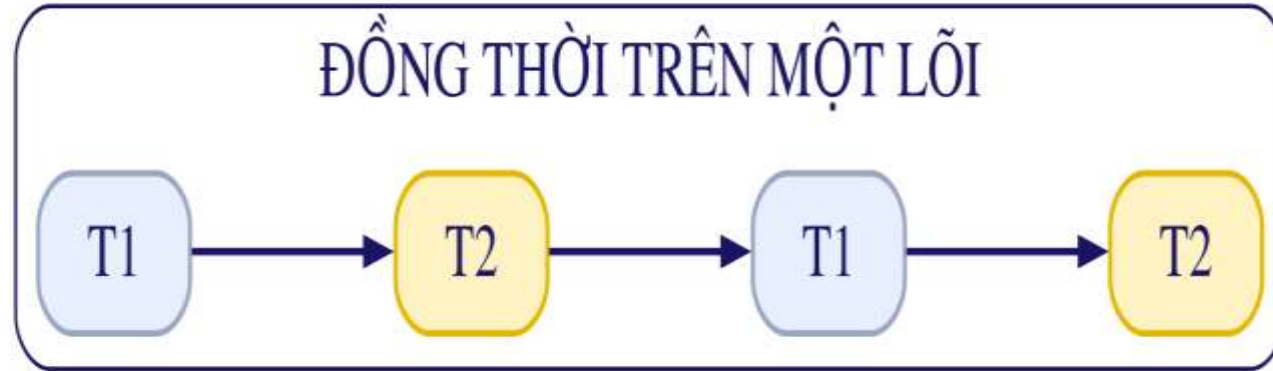


Ánh xạ Many-to-Many





Đồng thời không đồng nghĩa với song song





Tiến trình cách ly mạnh, luồng phối hợp nhẹ

Tiến trình

Có không gian địa chỉ và tài nguyên riêng; tạo/chuyển nặng hơn nhưng lỗi thường được cách ly tốt hơn. Giao tiếp cần IPC.

Luồng

Chia sẻ không gian địa chỉ và tài nguyên trong tiến trình; tạo/chuyển nhẹ hơn, giao tiếp trực tiếp nhưng lỗi có thể ảnh hưởng toàn tiến trình.

Chọn tiến trình khi cần cách ly và bảo vệ; chọn luồng khi các hoạt động liên quan cần chia sẻ dữ liệu và phối hợp hiệu quả.

Tổng kết Chương 3



Tiến trình là chương trình đang thực thi cùng trạng thái, ngữ cảnh và tài nguyên

Năm trạng thái cơ bản mô tả khả năng tiếp tục thực thi của tiến trình

Tạo và kết thúc tiến trình hình thành cây cha - con và đòi hỏi thu hồi tài nguyên

PCB lưu định danh, ngữ cảnh CPU, lập lịch, bộ nhớ, I/O và thông tin bảo vệ

Luồng là đơn vị sử dụng CPU; nhiều luồng chia sẻ tài nguyên của cùng tiến trình



Câu hỏi thảo luận

Vì sao tiến trình ở trạng thái Waiting không chuyển thẳng sang Running khi I/O hoàn tất?

PCB cần lưu những trường nào để chuyển đổi ngữ cảnh đúng đắn?

Khi nào nên dùng nhiều tiến trình thay vì nhiều luồng?

Email: minhnl@utc.edu.vn